



Blockchain Security Audit Report



Table Of Contents

1 Executive Summary

2 Audit Methodology

2.1 Compliance

2.2 Threat Modeling

2.3 Checklist

3 Project Overview

3.1 Project Introduction

3.2 Mainnet Assessment

4 Findings

4.1 Coverage

4.2 Vulnerability Information

4.3 Vulnerability Summary

5 Audit Result

6 Statement

1 Executive Summary

On 2026.07.03, the SlowMist security team received the NABD team's security audit application for nabd_chain, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "black, grey box lead, white box assists" to conduct a complete security test on the project in the way closest to the real attack.

The participants of this security audit are shown as follows:

Company	Auditor	Certification
SLOWMIST LIMITED	Jianhai Zhang	CISSP
SLOWMIST LIMITED	Changsheng Feng	CISSP

The test method information:

Test method	Description
Black-box testing	Verifies program behavior from the user perspective by observing outputs under different input scenarios, without access to internal implementation.
Gray-box testing	Designs test cases using a user perspective plus partial source-code awareness, focusing on inputs, outputs, and externally observable behavior.
White-box testing	Examines internal logic and implementation details in depth, including line-by-line analysis of critical code paths and potential defects.

The vulnerability severity level information:

Level	Description
Critical	Has a major impact on the security of the blockchain project and should be remediated immediately.

Level	Description
High	Significantly affects normal system operation and should be fixed as soon as possible.
Medium	Has a practical impact on project operation and should be scheduled for remediation.
Low	May affect system operation in specific scenarios and should be prioritized according to business context.
Weakness	Represents a theoretical security risk that is extremely difficult to reproduce in practice.
Suggestion	Does not necessarily constitute a direct vulnerability, but indicates better coding or architectural practices.
Information	Matches the intended design, but may still cause user asset loss or security misunderstanding under certain usage patterns.

2 Audit Methodology

2.1 Compliance

Ensure that the implementation of the blockchain complies with the regulatory compliance standards of major countries or regions.

2.2 Threat Modeling

SlowMist uses several classical security models to identify potential threats in cryptocurrency systems and to define audit priorities.

Model	Core Content	Description
CIA	Confidentiality, Integrity, Availability	Defines the core objectives of a security architecture.
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege	Helps systematically identify common threat categories and attack surfaces.

Model	Core Content	Description
DREAD	Damage Potential, Reproducibility, Exploitability, Affected Users, Discoverability	Supports risk prioritization for discovered vulnerabilities.
PASTA	Define objectives, scope the technology, decompose the application, analyze threats, identify vulnerabilities, enumerate attack paths, and assess risk and impact	Provides a threat analysis process aligned with architecture and business context.

2.3 Checklist

The security audit process of SlowMist security team for the chain includes two steps:

Chain codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.

Manual audit of the codes for security issues. The codes are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that was considered during the audit of the chain:

NO.	Audit Items	Result
1	Distributed Ledger Technology Evaluation	Passed
2	Security Infrastructure	Passed
3	Network Scale	Passed
4	Consensus Algorithm	Passed
5	Code Security	Some Risks
6	Historical Audit Records	Passed
7	Audit Institution Evaluation	Passed

3 Project Overview

3.1 Project Introduction

NABD is a permissioned, deterministic Byzantine Fault Tolerant (BFT) blockchain node written in Perl 5.42.0, engineered for institutional financial infrastructure. It combines hybrid post-quantum cryptography (Ed25519 + FN-DSA/FIPS 206 / Falcon-512), a custom 4-phase NabdBFT consensus engine, WASM3-sandboxed Autonomous Programs, multi-asset registry, ISO 20022 gateway, biometric verification, ZeroMQ P2P mesh, and native XS crypto/WASM modules.

3.2 Mainnet Assessment

Distributed Ledger Technology Evaluation

(1) Robustness Assessment

NabdChain (NABD) is a sovereign Layer-1 blockchain that employs deterministic BFT consensus (NabdBFT) with a self-healing liveness mechanism. The current public testnet runs $N=8$ validator nodes distributed across 4 validator virtual machines, with a quorum $Q=6$, tolerating Byzantine faults $f=2$, meaning that any single validator VM (containing two validator nodes) failure does not affect the continuous operation of consensus.

The system uses a heartbeat-driven recovery mechanism. The whitepaper and test data indicate a leader failover time of approximately 3.3 seconds. A view divergence/reseeding event occurred in June 2026, and a client-layer disk exhaustion event occurred on July 2, 2026. However, in both incidents, the consensus among validator nodes was unaffected, and blocks continued to be produced. The system includes a consensus liveness operations manual, deterministic replay scripts, backup/restore gating, and health check mechanisms as operational emergency safeguards.

Current known limitations: All validator nodes are hosted with a single cloud provider (Hetzner), with each VM running two validators. Provider concentration and regional concentration are disclosed risks. The public mode VRF leader election (based on libsodium draft-03 ECVRP) has been implemented but not yet activated, pending production

environment enablement. The production mainnet has not yet officially launched, and full robustness assessment will require independent validator admission and multi-provider deployment.

(2) Reliability Assessment

Regarding testnet reliability data, the repository recorded a low-throughput persistent stress test lasting 8 hours (completed on June 25, 2026): duration of 28,800 seconds, all 28,800 submitted transactions accepted, zero rejections, zero HTTP failures, zero state anomalies; maximum mempool sample was 0, maximum relay queue was 1; consensus view remained at 0, consecutive failures at 0; node peer connections 8/8; post-test full health gate passed on 8 validators, 2 clients, browsers, and load balancers.

Real-time verification data (July 3, 2026): all 8 validator nodes online and healthy, each validator with 7/7 peer connections, zero consecutive consensus failures, consensus rounds approximately 6,900,180, cumulative blocks 4,363,567, transactions 127,481. Average deterministic finality time was 1.214 seconds.

Publicly disclosed failure events include: the June 2026 view divergence incident (fixed and hardened) and the July 2, 2026 client/index layer stall due to disk exhaustion (validator consensus unaffected, repaired following storage archive runbook).

Security Infrastructure

(1) Integrity Mechanism

NabdChain employs a multi-layered guarantee mechanism for data integrity:

Hash Chain and Merkle Tree: Blocks and transaction/state data undergo deterministic hashing, with blocks carrying Merkle roots (lib/NABD/MerkleTree.pm). The codebase implements two Merkle hashing schemes—v1 (legacy) and v2 (domain-separated leaves/nodes and handling odd nodes). The current chain runs the v1 scheme, while v2 is available for future coordinated upgrades. Transaction paths use the BLAKE3 hash algorithm.

Digital Signatures: Transaction authorization supports Ed25519, Falcon-512, and hybrid signature schemes, using a scheme-aware signature envelope (signatures[]). Validators require that the public key of a valid signature must derive

the tx.from address. Consensus messages also require validator signatures. The system includes regression test coverage for signature substitution attacks (t/transaction_sig_substitution.t).

Chain Identity Binding: Each signed transaction is bound to a network_id and genesis_hash to prevent cross-chain replay attacks. Transactions also include nonce checks to prevent intra-chain replay, overdraft checks, and normalized JSON payloads to ensure deterministic serialization.

Deterministic Finality: After BFT quorum submission, finality is deterministic—there are no probabilistic confirmations or silent rollbacks. Any emergency recovery that changes finalized history requires explicit governance, operational procedures, and disclosure.

(2) Protection Capabilities

Consensus Layer Protection: NabdBFT adopts a four-phase protocol of PROPOSE, VOTE, CERTIFY, and COMMIT, with the quorum certificate serving as finality proof. Validator messages require signature verification, and the P2P layer has an identity checking mechanism. CurveZMQ configuration testing exists.

Smart Contract (Autonomous Program AP) Sandbox Isolation: APs execute as deterministic WASM3 modules via the XS interpreter path (not native JIT). The host environment enforces: manifest, owner, code hash, ABI hash, capability declaration, storage limit, fee limit, event limit, return size limit, timeout, memory boundaries, and upgrade/freeze policies. Deployment and invocation are signed transactions, constrained by nonce, replay protection, chain identity, and fee checks.

DDoS Protection: A layered protection strategy is adopted—reverse proxy TLS/rate limiting as prerequisites for deployment, with nginx rate limiting configurations present; at the node level, includes mempool size limit, relay queue limit, minimum fee, batch limit (default 250 transactions/request, hard cap 1,000 transactions), and backpressure thresholds. Public API and operational API are separated.

Operational Security: Administrative token gates operational/diagnostic endpoints; public API error messages are sanitized; wallet file permissions are protected (fail-closed); runtime fingerprint covers code, genesis, and consensus

parameters; production mode encrypts/WASM path behaves as fail-closed.

Post-Quantum Readiness: Supports the Falcon-512 signature scheme as a post-quantum authentication path, but explicitly notes it has not yet passed FIPS 206 verification; compliance will be completed after the final byte format and KAT are adopted.

MEV Risk: Under known validator topology, there is no public fee auction. Classic public mempool MEV (front-running via fee bidding) is structurally reduced, but validators and client operators can still observe transaction flows. Residual risks have been acknowledged and disclosed.

Network Scale

(1) Geographic Distribution

The current public testnet's 4 validator VMs are distributed across three Hetzner data center locations: fsn1 (Falkenstein, Germany), nbg1 (Nuremberg, Germany, two VMs), and hel1 (Helsinki, Finland).

Discrepancies disclosed: Whitepaper v9 marks the three sites as "Germany, Helsinki, Amsterdam," but the operations list does not include the Amsterdam site. The infrastructure documentation is the authoritative source, and the whitepaper will be corrected in the next revision.

Client/browser nodes are independent nodes outside the consensus quorum. The current geographic distribution covers only two European countries (Germany, Finland), with intercontinental distribution not yet achieved.

(2) Hosting Provider Distribution

The current public testnet validator node group is 100% hosted on the Hetzner cloud platform (CPX22 level VMs), using distributed placement groups (anti-affinity across physical hosts). The repository contains OCI Dubai deployment files, but these serve only as deployment/inventory examples and are not part of the active public testnet.

Multi-vendor operation is a planned expansion, and vendor concentration is explicitly recognized as a known risk.

(3) Degree of Centralization

The current public testnet features a known validator topology controlled by the project/foundation, not a permissionless staking pool market. Specific centralization risks are as follows:

- All validator nodes are operated by a single entity (NABD Foundation)
- A single cloud provider (Hetzner) hosts all validator infrastructure
- The alliance registry shows equal staking amounts for all validators (1,000,000,000,000 micro-NABD each)
- No liquid staking or validator delegation platforms
- Validator membership is determined by governance/operators, not open access
- Before mainnet, the following must be completed: multi-provider hosting, independent validator access program, geographically diversified deployment. If public staking is introduced on mainnet, it is necessary to add concentration thresholds, validator access rules, delegation caps, slashing/reward rules, and a disclosure dashboard.

Consensus Algorithm

(1) Consensus Algorithm Features

NabdChain uses NabdBFT—a deterministic Byzantine fault tolerance protocol with the following characteristics:

Protocol phases: A four-step process of PROPOSE → VOTE → CERTIFY → COMMIT. Confirmation is final upon achieving a quorum of commits, with no probabilistic finality.

Fault tolerance parameters: Quorum size $Q = \text{floor}(2N/3) + 1$, tolerance for Byzantine faults $f = \text{floor}((N-1)/3)$.

Currently, with $N=8$, $Q=6$ and $f=2$.

Leader election: In private/testnet modes, round-robin is used, coupled with heartbeat-driven recovery. In public mode, VRF-based leader election (based on libsodium draft-03 ECVRF) is implemented but gated, pending activation in production.

Finality performance: Measured average finality time of 1.214 seconds in testnet, with leader failover taking approximately 3.3 seconds.

Formal verification: A TLA+ formal model exists (under the `formal/tla/nabdbft/` directory: `NabdBFT.tla`, `NabdBFT_MC.tla`, `NabdBFT_MC_N7.tla`, and model configurations).

Throughput data: Whitepaper v9 records a single-wallet injection test with 4,700 transactions submitted with zero failures, achieving a confirmed TPS of 130 (84 blocks committed in 36.1 seconds). Earlier N=5 saturation tests show confirmed TPS ranging from 597 to 1,152 (average 936 TPS), kept only as historical capacity evidence.

(2) Consensus Implementation Security

Signature Security: Strict signature policy, transaction signatures are bound to the derivation of the sender's public key, and signature replacement attacks have regression test coverage. Consensus messages require validator signatures.

Deterministic Execution: Normalized JSON payload, deterministic ledger application, deterministic replay settlement gating, ensuring all nodes produce the same state for the same input.

Fail-closed Design: The production mode encryption path and WASM path adopt fail-closed behavior—default rejection instead of approval upon verification failure. Wallet permission checks are fail-closed, and validator staking checks are fail-closed.

Runtime Integrity: Runtime fingerprints cover code, genesis block, and consensus parameters, ensuring nodes run the expected version.

Test Coverage: At the time of review commit (f513e17), there are 114 test files covering key paths such as signature replacement, WASM allocator boundaries, transaction verification, and consensus flow.

No Slashing Market: The current testnet has no public slashing or staking penalty market; validator membership is a governance/operational decision. If a staking economic model is introduced on the mainnet, slashing rules need to be separately designed.

Code Security

The SlowMist security team conducted a black box and gray box audit on the key modules of Jovay, using automated tools and manual reviews, focusing mainly on the following areas:

- Static code analysis
- Encrypted and signature security audit
- Account and transaction security audit
- RPC Security audit
- P2P Security audit
- Consensus Security audit

Detailed audit details can be found in the following document:

<https://github.com/slowmist/Common-Cryptographic-Risks-in-Blockchain-Applications>

<https://github.com/slowmist/Cryptocurrency-Security-Audit-Guide/blob/main/Blockchain-Common-Vulnerability-List.md>

Different levels of safety risks were identified through manual assessment, details please refer to the findings section of this report.

Historical Audit Records

All existing audit-related documents of NabdChain are first-party (internal) review results.

Audit Institution Evaluation

N/A

4 Findings

4.1 Coverage

Audit scope:

https://gitlab.com/ktham33n1/nabd_chain

commit: f513e17faea6dabcf244227e1b5dee2694be4d70

4.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

NO	Title	Category	Level	Status
N1	Incorrect RPC Response Code	Code Security	Low	Confirming
N2	Nonce Gap Tolerance Default of 100 Enables Per-Sender Mempool Flooding (DoS)	Code Security	Low	Confirming
N3	Risk of Node Sensitive Information Leakage	Code Security	Information	Confirming
N4	MEV Attack Risks	Code Security	Information	Confirming
N5	Non-Trustless Blockchain	Code Security	Information	Confirming
N6	Merkle Tree v1 Root Collision via Odd-Node Duplication	Code Security	Suggestion	Confirming

4.3 Vulnerability Summary

[N1] [Low] Incorrect RPC Response Code

Category: Code Security

Content

Web child process crashes (500) when broadcasting transaction due to cleared signing keys, but the transaction actually succeeded on chain. PoC as follows: Request packet:

```
POST /api/v1/transactions HTTP/1.1
Host: 127.0.0.1:8081
User-Agent: curl/8.7.1
Accept: */*
Content-Type: application/json
Content-Length: 454

{"tx_hash":"622808c5931d605a4e26be76e61b5d5a5dcf83de8af6ea2c9ba80a137dd47033","to":"NABD1Q27G4CGWKPNRZ7KMC7D5H5FG6463CKV63G","pubkey":"97ccd8077e8ab18d1dd2e830e871abeccee1bcc42a7a99cf6654db052eee6899","nonce":1,"signature":"b8ce9a51435e372790c72a54dc34881f2d8f6d8772b8af96ab8769695c9cd09923b9dabc7331144f372f9efc99614597b45acf4cd863944911b11297ec3cec06","timestamp":1784100366,"from":"NABD1QFMVM6R4K8RKZLNJFCSHHXWJWAYQZPDHSJQ","amount":3000000,"fee":1000}
```

Response header:

```
HTTP/1.1 500 Internal Server Error
Content-Length: 32416
Content-Type: text/html; charset=UTF-8
Date: Wed, 15 Jul 2026 07:26:15 GMT
Server: Mojolicious (Perl)
Vary: Accept-Encoding
```

On-chain final state (transaction actually successful):

```
// GET
/api/v1/transactions/622808c5931d605a4e26be76e61b5d5a5dcf83de8af6ea2c9ba80a137dd47033
{
  "status": "confirmed",
  "block_round": 3031,
  "from": "NABD1QFMVM6R4K8RKZLNJFCSHHXWJWAYQZPDHSJQ",
  "to": "NABD1Q27G4CGWKPNRZ7KMC7D5H5FG6463CKV63G",
  "amount": 3000000, "fee": 1000, "nonce": 1
}

// GET /api/v1/accounts/NABD1Q27G4CGWKPNRZ7KMC7D5H5FG6463CKV63G
```

```
{
  "balance": 3000000, "total_received": 3000000, "tx_count": 1
}
```

If the client determines that the transaction failed based on the 500 status code and performs retry/compensation logic (such as rebuilding the transaction or assuming funds were not transferred), it may be inconsistent with the actual on-chain state, potentially leading to business reconciliation errors or user misjudgment of fund flow.

Solution

In validator mode, submitting transactions via the public HTTP API should return 200 + structured JSON, and should not result in a 500 error due to broadcast failure.

Status

Confirming

[N2] [Low] Nonce Gap Tolerance Default of 100 Enables Per-Sender Mempool Flooding (DoS)

Category: Code Security

Content

`TransactionPoolPersistent.pm:47-50` defaults `nonce_gap_tolerance` to 100 when `NABD_NONCE_GAP_TOLERANCE` is unset. A single sender can submit 100 pending transactions with non-sequential nonces (e.g., nonces 1–100 while expected is 0), exhausting per-sender mempool quota and inflating global pool size.

- lib/NABD/TransactionPoolPersistent.pm#L47-L50

```
my $nonce_gap_tolerance = exists $args{nonce_gap_tolerance}
    ? $args{nonce_gap_tolerance}
    : (defined $ENV{NABD_NONCE_GAP_TOLERANCE} ? $ENV{NABD_NONCE_GAP_TOLERANCE} :
100);
$nonce_gap_tolerance = 100 unless defined $nonce_gap_tolerance &&
$nonce_gap_tolerance =~ /^\\d+$/;
```

- lib/NABD/TransactionPoolPersistent.pm#L667

```
if ($nonce > $expected_nonce + ($self->{nonce_gap_tolerance} // 100)) {
    # reject
}
```

An attacker with a funded account submits 100 transactions with nonces 1–100 (skipping 0), filling 100 mempool slots and potentially exhausting the global `max_size` (default 10,000), preventing other users from submitting transactions.

Solution

Reduce `nonce_gap_tolerance` default to 5–10. Add per-sender transaction count limits in the mempool to cap a single account's contribution. Document `NABD_NONCE_GAP_TOLERANCE` in all deployment templates with a recommended safe value.

Status

Confirming

[N3] [Information] Risk of Node Sensitive Information Leakage

Category: Code Security

Content

`_public_api_safe_mode()` reads `NABD_PUBLIC_API_SAFE` with a default of `0` (disabled). In the default configuration, `/api/status` returns the full internal payload to any unauthenticated caller, including `consensus_leader`, `node_id`, `peers`, `mempool` size, `fingerprint`, and all `consensus_*` fields. The `?sqlite=1` parameter additionally exposes SQLite pragma settings.

- [nabd-full-node.pl#L2382-L2386](#)

```
sub _public_api_safe_mode {
    return _parse_bool($ENV{NABD_PUBLIC_API_SAFE}, 0); # Default: 0 (UNSAFE)
}
```

- [nabd-full-node.pl#L2498-L2522](#)

```
if (_public_api_safe_mode() && !_public_api_admin_authorized($c)) {
    # Redacted response -- only reached if NABD_PUBLIC_API_SAFE=1
    return $c->render(json => { network_id => ..., round => ... });
}
$c->render(json => $status_payload); # Full internal state exposed by default
```

Any external party can identify the current consensus leader, enumerate active peers, and monitor mempool size.

This enables targeted DoS against the leader, eclipse attacks, and timing-based transaction front-running.

Solution

Node operators need to be careful to prevent privacy data leaks.

Status

Confirming

[N4] [Information] MEV Attack Risks

Category: Code Security

Content

Maximal Extractable Value (MEV) attack risk arises from validators (or previously miners) exploiting their ability to reorder, insert, or censor transactions within blocks to extract additional value, often at users' expense. Common exploits include front-running (e.g., bidding higher gas to preempt trades), sandwich attacks (surrounding a user's transaction to profit from price slippage), and arbitrage manipulation in DeFi protocols. This stems from the transparent mempool and block production incentives, potentially leading to unfair advantages and economic inefficiencies.

Solution

Mitigation strategies include Proposer-Builder Separation (PBS) for fairer auctions, encrypted mempools, and user-side protections like private RPCs to minimize exposure.

Status

Confirming

[N5] [Information] Non-Trustless Blockchain

Category: Code Security

Content

NabdChain (NABD) is a sovereign Layer-1 blockchain that employs deterministic BFT consensus, exhibiting a degree of centralization and requiring users to trust the project operator.

The 8 validators are distributed across only 4 hosts (2 per host), all under the same operator/cloud account/private subnet. BFT nominally tolerates 2 Byzantine faults, but the failure of 2 hosts would result in the loss of 4 validators, breaking the survival quorum and causing a halt; a single entity holding all the keys could unilaterally reorganize or censor the chain.

Solution

At most 1 validator per host, distributed across availability zones/entities; expand the set and introduce independent operators.

Status

Confirming

[N6] [Suggestion] Merkle Tree v1 Root Collision via Odd-Node Duplication

Category: Code Security

Content

The Merkle tree implementation defaults to schema version 1 (legacy). In v1, when the number of leaves is odd, the last node is duplicated and hashed with itself. This is the well-known CVE-2012-2459 pattern, allowing two distinct transaction sets to share the same Merkle root. Schema v2 (which fixes this by promoting the lone node unchanged) exists in the code but is never activated: all genesis configurations lack the `merkle_schema` field, so

`set_merkle_schema(undef)` always resolves to schema 1.

- lib/NABD/MerkleTree.pm#L92-L105

```
# v1 (legacy, preserved for existing blocks): duplicate the last
# node and hash it with itself.
push @next_level,
  _node_hash( $schema, $current_level->[$i], $current_level->[$i] );
```

Two transaction sets [A, B, C] and [A, B, C, C] can produce identical Merkle roots, allowing an attacker to forge SPV/light-client membership proofs. Any verifier that trusts the Merkle root without replaying the full transaction list is deceived.

Solution

Activate Merkle schema v2 by adding `"merkle_schema": 2` to `config/genesis.json` and `docker/genesis.json`. Add a startup assertion that `schema >= 2` in production mode. Provide a migration path (schema upgrade transaction type) for existing deployed nodes.

Status

Confirming

5 Audit Result

Audit Number	Audit Team	Audit Date	Audit Result
0X002607160001	SlowMist Security Team	2026.07.03 - 2026.07.16	Low Risk

Summary conclusion: The SlowMist security team use a manual and SlowMist team's analysis tool to audit the project, during the audit work we found 2 low risk, 1 suggestion vulnerabilities. All the findings were fixed. The code was not deployed to the mainnet.

6 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.



Official Website
www.slowmist.com



E-mail
team@slowmist.com



Twitter
[@SlowMist_Team](https://twitter.com/SlowMist_Team)



Github
<https://github.com/slowmist>